istic Informatique Électronique

M2 Ingénierie des Réseaux
Université de Rennes 1

Rapport de projet du semestre 1, phase 2
Responsable de projet : Djamal Meddour

Implémentation d'un mécanisme de transmission de vidéo multicast sur un réseau sans fil

F. DOUSSINET, D. LE QUÉRÉ, J. MARECEK

Rennes, 14 février 2011

Chapitre 1

Rappel du problème et solution choisie

1.1 Problématique

L'expansion des réseaux sans fil (WLAN [1]) durant ces dernières années n'est plus à démontrer. Dans ce contexte, la solution Wi-Fi [2] centralisée est la solution actuellement prépondérante en terme de déploiement sur les réseaux *mesh* ou *ad hoc*. Ses particularités par rapport à un réseau local filaire classique sont d'ordre structurelle : on passe systématiquement par une station particulière (station de base ou Access Point) pour communiquer, ce qui amène les inconvénients d'une topologie en étoile ; et physique : la qualité du signal reçu est variable entre les stations d'un même réseau local selon leur éloignement avec leur point d'accès et les perturbations de l'environnement (bruit).

Étant donné cette particularité physique des réseaux Wi-Fi, et étant donné les services attendus sur ces réseaux que l'interconnexion avec internet implique, il est intéressant de poser la question de la qualité de service sur ce type de réseau, en particulier la transmission de flux multimédia.

Le problème de notre projet est celui de la qualité de la diffusion vidéo sur les réseaux sans fil de type Wi-Fi. Il s'agit de mettre en place un mécanisme de diffusion vidéo multicast qui prend en compte la variabilité de la qualité du signal reçu par les stations sur un tel réseau. Concrètement, avec une diffusion vidéo multicast "classique", l'utilisation d'un débit élevé pénalise les utilisateurs les plus éloignés du point d'accès et un débit faible pénalise les plus proches (*figure 1.1*).

Il est donc question de trouver un mécanisme de transmission avec un débit différencié en fonction du terminal. Pour l'implémentation d'une telle solution, il faudra prendre en compte la **qualité du signal reçu** par l'utilisateur final et son adéquation par rapport à la vidéo transmise. Une extension de ce mécanisme pourra permettre par la suite d'adapter la vidéo transmise en fonction du type de terminal : ordinateur portable, smart phone, PDA etc.

Il nous est proposé d'utiliser un codage scalable débits de la source vidéo avec H264/SVC et de transmettre au terminal la vidéo qui correspond au mieux à ses contraintes vidéo. En pratique, il s'agit de travailler sur l'implémentation MadWifi de la couche MAC 802.11 (*figure 1.2*) afin de

1. Wireless Local Area Network
2. Wi-fi en mode infrastructure

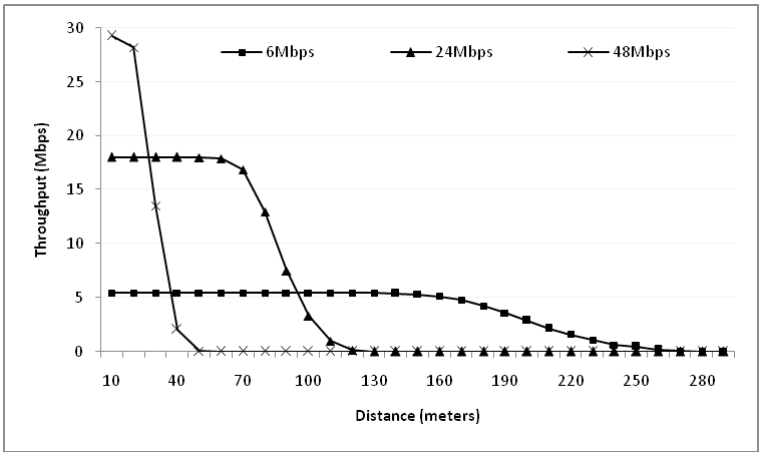


FIGURE 1.1 – Mesure du débit en fonction de la distance avec le point d'accès

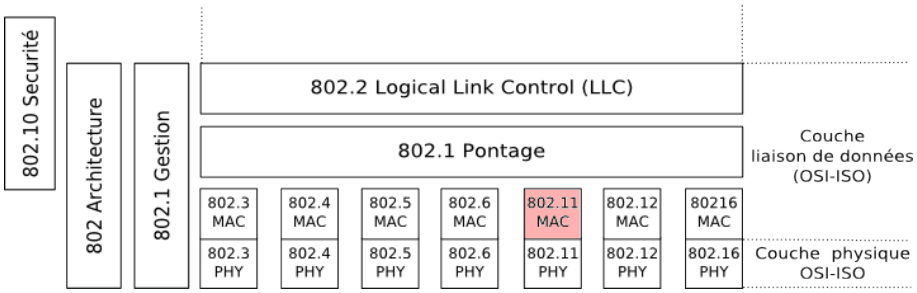


FIGURE 1.2 – La couche MAC 802.11 dans la norme IEEE 802

récupérer les informations relatives à la qualité du signal par rapport à un terminal donné, ensuite **faire une transmission qui correspond au débit approprié** (*figure 1.3*).

1.2 Mécanismes à mettre en place

D'abord, pour des raisons de temps et de taille du groupe de travail, le mécanisme de récupération de la qualité du signal d'un terminal a été mis de côté à l'issue de la première phase du projet au profit du mécanisme de transmition de débits différentiés qui constitut l'essentiel du travail. Ensuite, la mise en place pratique du système de diffusion nous a contraint à remettre en cause deux choix établis durant la première phase.

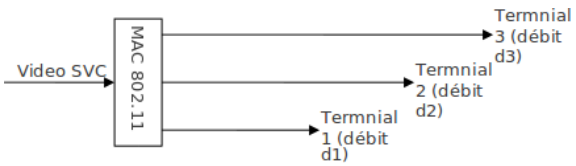


FIGURE 1.3 – On adapte le débit vidéo au niveau de la couche MAC avec MadWifi

```
if estNAL(paquetCourant) then
  if estNALVCL(paquetCourant) then
    if (paquetCourant.couche = baseLayer.couche) then
      envoyer(paquetCourant, débitMin)
    else
      envoyer(paquetCourant, débitMax)
    end if
  else
    envoyer(paquetCourant, débitMin)
  end if
end if
```

FIGURE 1.4 – Algorithme de débit différencié avec deux niveaux de débits (Min et Max)

Travail au dessus d'IP :

Nous prévoyions, pendant la première phase du projet, de diffuser le flux vidéo directement aux niveau liaison de données du modèle OSI car nous imaginions un serveur de diffusion ne fonctionnant que localement. Nous avons rapidement remis en cause ce choix quand nous nous sommes rendu compte que nous ne pourrions trouver de serveur de diffusion fonctionnant au niveau 2 du modèle OSI. Nous nous sommes donc dirigé vers des solutions fonctionnant "au dessus d'IP". Ce qui nous a fait passer d'un travail de diffusion sur un groupe multicast Ethernet à un groupe multicast IP.

Mécanisme de transmission de débits différentiés :

Nous prévoyions, pendant la première phase du projet, d'envoyer trois flux vidéos sur trois adresses multicast distinctes. Comme on nous l'a fait remarquer, cette solution pose le problème de la synchronisation des paquets et rend le mécanisme complexe. D'autre part, cette solution est difficilement extensible car le nombre d'adresses multicast et leur association avec les niveaux de qualité est fixée selon le système de diffusion souhaité (ie le nombre de niveaux de qualité voulu) et elle impacte à la fois la configuration du serveur et celle des clients.

Nous avons donc opté pour une solution plus tranparente pour le client : on envoi un flux vidéo scalable SVC sur une seule adresse multicast, mais on va faire varier la modulation du signal selon l'importance des paquets. Les paquets NAL VCL correspondants à la couche de base du codage SVC seront envoyés avec une modulation robuste [3] de même que les paquets NAL non-VCL qui contiennent des informations indispensables pour la présentation. On enverrera par contre les NAL VLC des couches d'amélioration de SVC sur des modulations moins robustes mais avec un meilleur débit comme le montre l'algorithme de la *figure 1.4*

Ainsi, la couche de base (Base Layer) est transmise sur une plus grande distance par rapport au point d'accès et va être plus robuste aux perturbations du signal. Ce qui fait que, indépendamment du type de terminal, on reçoit un flux décodable plus loin qu'avec un système de diffusion non différencié tout en accédant à une meilleur qualité vidéo à mesure que l'on obtient un meilleur signal. On peut remarquer en plus que par rapport à la solution avec plusieurs flux multicast,

3. **NB :** On parle ici de *modulation*, ce qui se traduira dans MadWifi par une variation du *un débit* car nous somme au niveau de la couche MAC 802.11 et non au niveau de la couche PHY (*figure 1.2*)

l'adapation de la qualité est immédiate puisque le mécanisme est uniquement basé sur un traitement côté serveur au niveau MAC, le client se contente de décoder le flux vidéo selon les paquets qu'il a été en mesure de recevoir.

NB : On parle ici de *modulation*, ce qui se traduira dans MadWifi par une variation du *un débit* car nous somme au niveau de la couche MAC 802.11 et non au niveau de la couche PHY (*figure 1.2*).

Chapitre 2

Mise en œuvre

Durant la phase 2 du projet, nous avons consacré la plus grande partie de notre temps à la mise en place du système de diffusion : envoi d'un flux multicast H264/SVC sur un réseau local sans fil. Il nous a semblé indispensable de faire fonctionner complètement la chaîne de traitement [1] avant de commencer les modifications du driver *MadWifi* pour nous permettre de travailler sur les trames réelles à traiter dans la partie d'adaptation du driver *MadWifi*.

2.1 Configuration du système de diffusion

Cependant, nous avons rencontré des problèmes techniques et avons mis du temps à trouver le serveur de diffusion et le lecteur/décodeur que nous cherchions pour H264/SVC, et nous n'avons pas obtenu un système fonctionnel de bout en bout.

Néamoins nous avons pu :

1. configurer un réseau Wi-Fi en mode managed (AP/Station) [**cartes atheros, MadWifi**]
2. encoder un fichier SVC selon le niveau de scalabilité voulu [**JSVM**]
3. diffuser un flux H264/SVC en multicast avec le serveur de diffusion **testH264VideoStreamer** fournit avec le serveur de streaming **live555** (ce serveur est le seul que nous ayons trouvé capable de *diffuser* un flux H264/SVC). Le protocole de transport utilisé est UDP
4. nous abonner au groupe multicast côté client [**programme C**]
5. lire un fichier H264/SVC localement (sur le disque) avec le lecteur **Mplayer** recompilé avec l'option `./configure --enable-svc` pour lui ajouter la fonctionnalité de décodage H264/SVC

La diffusion ne fonctionne pas de bout en bout : lors de la lecture du flux, *Mplayer* nous affiche l'erreur suivante : `Streem not seekable`. Nous précisons dans les paragraphes suivants certains aspects de la configuration qui nous semble intéressants pour mettre en place le système (voir également *figure 2.1*)

1. *encodage* du fichier SVC, *transmission* du flux, abonnement multicast côté client puis *décodage* et *lecture* du flux vidéo

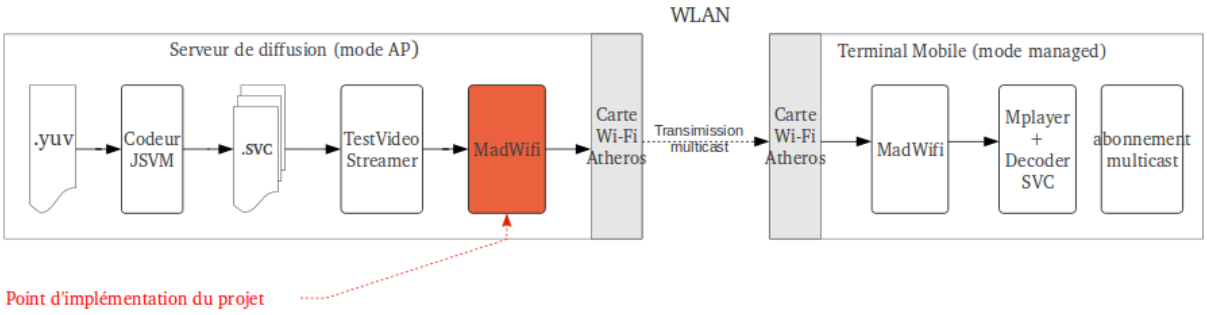


FIGURE 2.1 – Composants de notre système de diffusion

Configuration du réseau sans fil en mode managed

Voila ci-dessous les commandes principales pour utiliser *MadWifi* une fois qu'il est installé (fichiers bash de configuration en annexe). Il faut bien sûr commencer par configurer l'AP pour pouvoir ensuite lui rattacher une station.

Commandes Unix Serveur (AP) :

```
 # attention : adapter le nom de l'interface (athN) selon votre carte
sudo su # on commence par passer en mode root
rmmod ath5k # on désactive le driver en cours d'utilisation
/etc/init.d/network-manager stop # on stop network-manager
killall wpa_supplicant # on tue le processus wpa_supplicant qui peut gener la configuration
modprobe ath_pci # on active le driver madwifi
wlanconfig ath0 destroy # on détruit l'interface athx
wlanconfig ath0 create wlandev wifi0 wlanmode ap # on crée l'interface athx en mode AP
iwconfig ath0 essid projet_svc # on choisi l'ESSID de l'AP
ifconfig ath0 192.168.100.1 netmask 255.255.255.0 up # on donne une adresse IP à athx et on l'active
```

Commandes Unix Client (station) :

```
 # attention : adapter le nom de l'interface (athN) selon votre carte
sudo su # on commence par passer en mode root
/etc/init.d/network-manager stop # on stop network-manager
killall wpa_supplicant # on tue le processus wpa_supplicant qui peut gener la configuration
rmmod ath5k # on désactive le driver en cours d'utilisation
modprobe ath_pci # on active le driver madwifi
iwconfig ath2 essid projet_svc # on précise le nom du ESSID auquel on se ratache
ifconfig ath2 192.168.100.2 up # on donne une adresse IP à ath2 et on l'active
iwconfig ath2 ap auto # on se ratache à notre AP de façon automatique
route add -net 232.0.0.0 netmask 255.0.0.0 dev ath2 # on ajoute des routes multicast vets ath2
```

Encodage JSVM

Configuration générale *JSVM* est une bibliothèque d'applications libre de droits et multi-plateforme, qui propose plusieurs outils pour manipuler des fichiers H264/SVC. Parmi ces outils, nous avons surtout utilisé le programme *H264AVCEncoderLibTestStatic* avec lequel on peut, à partir d'un seul ou plusieurs fichiers vidéo (.yuv), encoder un fichier H264/SVC. Nous nous sommes aussi servi de *BitStreamExtractorStatic* pour afficher des informations sur les couches d'un fichier H264/SVC.

Pour utiliser le programme *H264AVCEncoderLibTestStatic* il faut lui passer en paramètre le fichier principal de configuration *main.cfg*. (`H264AVCEncoderLibTestStatic.exe -pf main.cfg`) Ce fichier permet de paramétrer précisément l'encodage du programme, en voici les principales options :

```
 #============================== GENERAL ======================================
AVCMode 0 # doit être égal à 0 pour le mode SVC
OutputFile test.264 # fichier de sortie
FrameRate 30.0 # nombre maximal d'images par seconde [Hz]
FramesToBeEncoded 100 # nombre d'images à encoder

#============================== LAYER DEFINITION ==============================
NumLayers 2 # nombre de couches
LayerCfg layer0.cfg # fichier de configuration d'une couche
LayerCfg layer1.cfg # fichier de configuration d'une couche
```

On remarque que la dernière partie contient les liens vers les autres fichiers de configuration des différentes couches du fichier encodé en H264/SVC.

Configuration d'une couche Un fichier de configuration d'une couche se présente sous la même forme que le fichier principal. Il contient lui aussi un ensemble d'options pour paramétrer le codage de la couche.

```
 #====================== INPUT / OUTPUT ========================================
SourceWidth 176 # largeur d'une image
SourceHeight 144 # hauteur d'une image
FrameRateIn 15 # nombre d'images par seconde en entré
FrameRateOut 15 # nombre d'images par seconde en sorte
InputFile orig.yuv # fichier d'entré
ReconFile rec_layer.yuv # fichier de reconstruction (mod debug)
```

Informations sur les couches *BitStreamExtractorStatic* qui permet de connaître le nombre de couches d'un fichier H264/SVC. (`BitStreamExtractorStatic input.h264`)

```
 Layer Resolution Framerate Bitrate DTQ
0 176x144 3.7500 106.00 (0,0,0)
1 352x288 3.7500 666.00 (0,1,0)
2 352x288 3.7500 1010.00 (0,2,0)
```

L'information principale est le triplet DTQ (Dependency_id, Temporal_id et Quality_id) qui indique respectivement les niveaux d'amélioration spatiale, temporel et de la qualité. Grâce à ce tableau, nous sommes donc en mesure de déterminer de façon précise à quelle couche appartient les NALs (Network Abstraction Layer) qui compose le flux vidéo codé en SVC.

2.2 Adaptation du driver MadWifi

La mise en place de l'algorithme de débit différencié (*figure 1.4*) est facile à comprendre : il faut récupérer dans MadWifi les informations nécessaires pour déterminer trois choses :

- le paquet courant est-il un NAL unit ? (c'est à dire un paquet H264/SVC)
- si le paquet est un NAL, est-il VCL ou non ?
- si le paquet est un NAL VCL, à quelle couche correspond-t-il ?

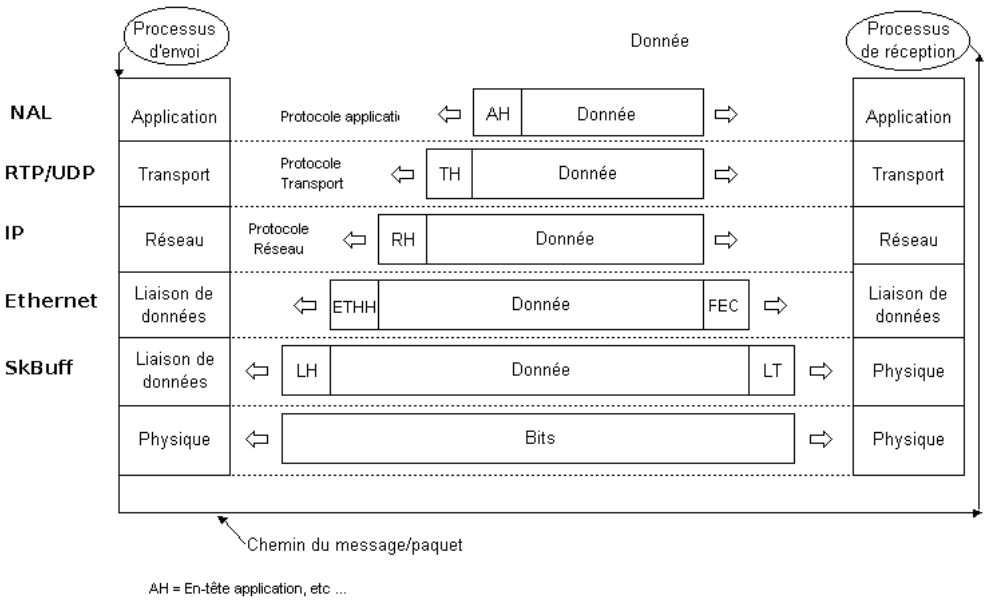


FIGURE 2.2 – Encapsulation des paquets dans MadWifi

MadWifi est un driver Wifi qui agit au niveau MAC 802.11, ainsi, si la structure des paquets *skbuff* de MadWifi peut nous permettre d'obtenir les adresses ethernet de la source et de la destination, les données contenues ne sont pas structurées car elles sont traitées par les couches supérieures.

Or, fonctionnant au dessus de IP, notre mécanisme va d'abord consister à structurer les données récupérées et récupérer les informations décrites plus haut. On distinguera donc deux phases dans la mise en place du mécanisme : **la structuration des données** et la **mise en place de l'algorithme de débit différencié** proprement dit.

La structuration des données

MadWifi se situant au niveau liaison de données du modèle OSI, pour récupérer les informations qui nous intéressent nous devons restructurer les données depuis la trame Ethernet jusqu'aux structures NAL unit (*figure 2.2*).

Algorithme de débits différencié

Avant de se décider concrètement sur les modifications à apporter au code C de MadWifi, nous devions définir l'endroit où les effectuer. L'emplacement choisi est la fonction *ath_hardstart*, elle est appelée périodiquement (toutes les 20 ms environ) et elle transmet les paquets. Avec cette fonction, nous avons accès aux deux parties principales de notre projet :

- la structure sk_buff, qui contient les données encapsulées NAL_VCL et NAL_NON_VLC,
- la fonction *ath_tx_start()* pour la gestion des débits.

La structure sk_buff.h La structure sk_buff (*figure 2.3*) est déclarée dans le fichier `skbuff.h` du répertoire `linux`. Nous nous sommes concentrés sur 4 attributs : unsigned char ∗head,∗data,∗tail,∗end.

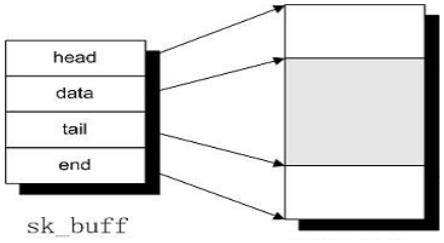


FIGURE 2.3 – La structure skbuff

L'attribut *head* pointe sur l'entête de la trame Ethernet, *data* pointe sur les données contenues dans le paquet et *tail* et *end* pointent respectivement sur la fin des données et sur la fin de la trame Ethernet. Grace à ces pointeurs, nous pouvons décapsuler chaque trame pour obtenir à son tour le paquet IP qu'elle contient. Et ainsi de suite jusqu'aux NALs souhaités. Pour la structuration des paquets IP, nous avons ajouté la structure d'un paquet IPv4 et développé une fonction qui effectue la segmentation et remplie cette structure.

La gestion du débit Il faut savoir que MadWifi, tel que nous l'avons configuré, choisi automatiquement le meilleur débit parmi tous les débits supportés par le matériel pour envoyer les données. Pour faire ce choix, il utilise un tableau qu'il initialise avec des données systèmes et fait correspondre chaque débit avec un code. Ce tableau est accessible via la fonction *ath_tx_findindex(...)* qui prend comme paramètre le tableau de débits et retourne un code associé au débit choisi.

Afin de définir précisément quel débit offrir à tel NAL, un travail préliminaire doit être effectué. Grace à l'application *BitStreamExtractorStatic*, nous pouvons connaître le nombre de couches d'un fichier H264/SVC en lien avec le triplet <D,T,Q>. Avec un appel système `iwlist [interface] rate` nous pouvons connaître les débits disponnibles sur l'interface radio.

Nous pouvons créer un tableau fixe qui fait le lien entre un débit et un triplet.

Lors de l'appel à la fonction *ath_tx_start(...)* on lui passe une structure *ieee80211vap vap* qui contient un attribut *int iv_mcast_rate* représentant le débit du paquet.

En utilisant le tableau précédent en fonction du triplet <D,T,Q> extrait du NAL, on peut modifier cette valeur pour lui attribuer un nouveau débit. La fonction *ath_hardstart(...)* continue normalement son exécution et fait appel à *ath_tx_findindex(...)* qui retournera le code du débit choisi par nos soins.

Chapitre 3

État du projet et perspectives

Nous n'avons pas pu terminer la mise en place du mécanisme de diffusion H264/SVC avec des débits différenciés par couche. Nous avons quasiment mis en place le système de diffusion en installant et en configurant les outils. En revanche, par manque de temps, nous n'avons pas pu terminer la phase de développement dans MadWifi pour la mise en place de l'algorithme de débits différenciés. Sur ce point nous nous sommes arrêté après la segmentation des paquets au niveau de IP.

Pour poursuivre ce projet, il s'agit donc de terminer la partie configuration pour être capable de faire fonctionner la diffusion de bout en bout et de continuer la mise en place de l'algorithme.

Bibliographie

Documents initiaux

- *A Technical Tutorial on the IEEE 802.11 Protocol*
  by : Pablo Brenner, Director of Engenieering
- *Overview of the Scalable Video Coding Extension of the H.264/AVC Standard*
  by : Heiko Schwarz, Detlev Marpe, Member, IEEE, and Thomas Wiegand, Member IEEE

Mémoires

- *Diffusion Video Scalable en Pair-A-Pair*
  par : Amer Abdhelhalim
- *Architecture en 802.11*
  par : Eric Bouche, 2007

Sites Web

- MadWifi :
  `http://madwifi-project.org/`
- Encoder JSVM :
  `http://ip.hhi.de/imagecom\_G1/savce/downloads/SVC-Reference-Software.htm`
- Commandes Linux :
  `http://webmail.appert44.org/~plegal/index.php/Configuration_et_test_d%27une_interface_r%C3%A9seau_sur_une_station_GNU/Linux`

Tutoriaux

- MadWifi :
  `http://wireless.poly.edu/wiki/Madwifi%20Tutorial`
- Encoder JSVM :
  `http://ube.ege.edu.tr/~boztok/JSVM/SoftwareManual.pdf`
- Serveur de diffusion Live555 :
  `http://www.live555.com/`
- Lecteur de flux vidéo :
  `http://www.mplayerhq.hu/design7/news.html`

Annexe A

Scripts et programmes de configurations

config_serveur.sh

```
#!/bin/bash

sudo rmmod ath5k
sudo service network-manager stop
sudo killall wpa_supplicant
sudo modprobe ath_pci
sudo wlanconfig ath0 destroy
sudo wlanconfig ath0 create wlandev wifi0 wlanmode ap
sudo iwconfig ath0 essid projet_svc
sudo ifconfig ath0 192.168.100.1 netmask 255.255.255.0 up
sudo route add -net 232.0.0.0 netmask 255.0.0.0 dev ath0

iwconfig ath0
ifconfig ath0
route -n

echo "
Configuration terminée
"

#Configuration DEBUG
sudo athdebug +xmit +rate

exit 0
```

config_client.sh

```
#!/bin/bash

sudo /etc/init.d/network-manager stop
sudo killall wpa_supplicant
sudo rmmod ath5k
sudo modprobe ath_pci
sudo iwconfig ath2 essid projet_svc
sudo ifconfig ath2 192.168.100.2 up
#sudo [iwlist ath2 scanning]
sudo iwconfig ath2 ap auto
sudo iwconfig ath2 ap any
sudo ifconfig ath2 192.168.100.2
sudo route add -net 232.0.0.0 netmask 255.0.0.0 dev ath2

echo "Configuration terminée"
```

```
IWCONFIG ATH2"
iwconfig ath2

echo "IFCONFIG ATH2"
ifconfig ath2

echo "Routage :"
route -n

echo "Access points visibles :"
iwlist ath2 ap

echo "Ping de l'AP :"
sudo ping 192.168.100.1 -c 3

exit 0
```

reinitialisation.sh

```
#!/bin/bash
# Attention remplacer athN par l'interface utilisée !

sudo rmmod ath_pci
sudo modprobe ath5k
sudo service network-manager start

iwconfig athN
ifconfig athN

exit 0
```

abonnement_multicast.c

```
/*
 * listener.c -- joins a multicast group and echoes all data it receives from
 * the group to its stdout...
 */

#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <time.h>
#include <string.h>
#include <stdio.h>


#define HELLO_PORT 12345
#define HELLO_GROUP "232.222.22.2"  // ADRESSE DU GROUPE MULTICAST
#define MSGBUFSIZE 256

main(int argc, char *argv[])
{
     struct sockaddr_in addr;
     int fd, nbytes,addrlen;
     struct ip_mreq mreq;
     char msgbuf[MSGBUFSIZE];

     u_int yes=1;            /*** MODIFICATION TO ORIGINAL */

     /* create what looks like an ordinary UDP socket */
     if ((fd=socket(AF_INET,SOCK_DGRAM,0)) < 0) {
  perror("socket");
  exit(1);
     }
```

```
/**** MODIFICATION TO ORIGINAL */
    /* allow multiple sockets to use the same PORT number */
    if (setsockopt(fd,SOL_SOCKET,SO_REUSEADDR,&yes,sizeof(yes)) < 0) {
       perror("Reusing ADDR failed");
       exit(1);
       }
/*** END OF MODIFICATION TO ORIGINAL */

     /* set up destination address */
     memset(&addr,0,sizeof(addr));
     addr.sin_family=AF_INET;
     addr.sin_addr.s_addr=htonl(INADDR_ANY); /* N.B.: differs from sender */
     addr.sin_port=htons(HELLO_PORT);

     /* bind to receive address */
     if (bind(fd,(struct sockaddr *) &addr,sizeof(addr)) < 0) {
  perror("bind");
  exit(1);
     }

     /* use setsockopt() to request that the kernel join a multicast group */
     mreq.imr_multiaddr.s_addr=inet_addr(HELLO_GROUP);
     mreq.imr_interface.s_addr=htonl(INADDR_ANY);
     if (setsockopt(fd,IPPROTO_IP,IP_ADD_MEMBERSHIP,&mreq,sizeof(mreq)) < 0) {
  perror("setsockopt");
  exit(1);
     }

     /* now just enter a read-print loop */
     while (1) {
  addrlen=sizeof(addr);
  if ((nbytes=recvfrom(fd,msgbuf,MSGBUFSIZE,0,
       (struct sockaddr *) &addr,&addrlen)) < 0) {
       perror("recvfrom");
       exit(1);
  }
  //puts(msgbuf);
     }

}
```

Table des figures

1.1 Mesure du débit en fonction de la distance avec le point d'accès 4
1.2 La couche MAC 802.11 dans la norme IEEE 802 4
1.3 On adapte le débit vidéo au niveau de la couche MAC avec MadWifi 4
1.4 Algorithme de débit différencié avec deux niveaux de débits (Min et Max) 5

2.1 Composants de notre système de diffusion 8
2.2 Encapsulation des paquets dans MadWifi 10
2.3 La structure skbuff 11

Table des matières

1 Rappel du problème et solution choisie **3**
1.1 Problématique 3
1.2 Mécanismes à mettre en place 4

2 Mise en œuvre **7**
2.1 Configuration du système de diffusion 7
2.2 Adaptation du driver MadWifi 9

3 État du projet et perspectives **12**

A Scripts et programmes de configurations **14**